

Fair Intelligence CMS & Role Modelling Plan

Oct 3, 2026 · @Refal

Summary

Fair Intelligence runs three roles on any workspace: PR, Brand & KOL and Social Media. This plan (v2, revised after Refal's review on 4 Oct) turns them into products that Fair owns, improves and releases, and that every client can make its own.

- **The CMS is Fair's brain, and only Fair uses it.** Refal and Rafli own it. The product team improves the roles there; data ops sets up each client's workspace and keeps it healthy. Clients never open it.
- **Each role is a Fair product with a codename and a version: Chorus (PR), Atlas (Brand & KOL) and Spark (Social Media).** Today they are Chorus 1.0, Atlas 1.0 and Spark 1.0. Audia, Wega and Arneta own them together for now.
- **The Role Lab is how the team improves a role fast.** A role owner works with AI inside the CMS, the way Refal works with Claude today: ask for an improvement, let the AI build it into a draft, run the role's tests on real workspaces, look at the result, iterate. Only Refal and Rafli release a version; role owners can roll one back.
- **Every client gets its own version of each role.** The client's **Builder** shapes the company-wide version ("GoPay's Chorus"), mostly by asking CeMO. **Members** use it daily and create their own skills, decks and reports, which go live after the Builder approves. Everything carries a badge: **Fair** for what came with the role, the client's name for what they made.
- **Clients can extend their data, never change the core.** A Builder can add a Persona table or their own metrics through CeMO. The core tables stay the same for everyone; extensions sit on top for that client only, are paid in credits, and the good ones can later be promoted into the core.
- **Everything is recorded, and the roles learn.** What people make, approve, keep, change and ignore flows back in three steps, Member → Builder → Fair. Role owners see it in the CMS and turn it into the next version.

Two rules hold throughout, both from CLAUDE.md. Nothing is shared across workspaces: learning moves *shapes* (settings, layouts, thresholds), never data, text or numbers from one workspace into another. And the model never computes numbers: a role model can change what is shown and how CeMO speaks, but every figure still comes from SQL in skills, the dashboard and the deck facts.

Delivery is seven phases, about seventeen weeks of build. It starts with the foundation: role versions in tables with their codenames, releases and rollback, and no visible change for clients. The decisions still open are at the end.

One brain, three layers

The product has two sides. **Fair's side** is the CMS, where the roles are made, tested and released, and where every client workspace is set up and watched. **The client's side** is the product itself, where a client's Builder and Members use a role and make it their own. A role reaches a client in three layers, and the effective role is resolved on every request.

Layer 1 — The Fair role (owned by Fair)

One per role: Chorus, Atlas, Spark, and any role added later. It is Fair's best current answer to "what does a PR team need": its dashboard, skills, deck templates, CeMO's voice, signals and vocabulary, plus the test set that proves it works. It is versioned (Chorus 1.0, 1.1, 2.0), improved in the Role Lab, and released only by Refal or Rafli. Each part carries a **policy** that says who may change it below:

- **Locked** — only Fair. Examples: the status ladder's levels and their order, the rule that issue labels compare shares not counts, the comment floor below which no status is shown, the definition of a measured post, the core tables.
- **Builder-editable** — the client's Builder may change it for the whole company: thresholds within guard rails, which tiles show, deck templates on offer, CeMO's house rules, vocabulary.
- **Member-editable** — each Member may set it for themselves: default window, brand in focus, tile order, which alerts reach them, answer length.

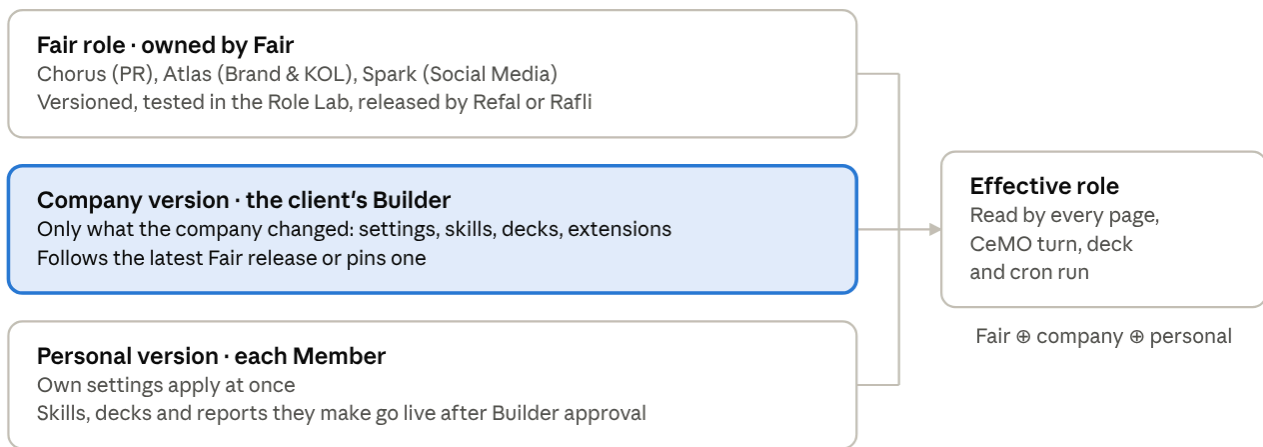
Layer 2 — The company version (owned by the client's Builder)

One per workspace × role: "GoPay's Chorus". It stores only what the company changed, never a full copy. It holds the Builder's changes to Builder-editable parts, the company's own skills and deck templates, its house rules and memory, the data extensions it added, and which Fair version it sits on (following the latest release, or pinned to one).

Layer 3 — The personal version (owned by each Member)

One per person × workspace × role. Personal settings apply at once. Anything a Member *makes* (a skill, a deck template, a report format, a house rule) is a draft until the Builder approves it; once approved it joins the company version for everyone, with the Member named as its maker.

Each client runs Fair's role plus its own changes, resolved on every request



A lower layer changes a part only where that part's policy allows it

role layers · Fair, company, personal

The company layer is the new part: it holds only what a client changed, so a new Fair release still reaches everything the client left alone. Clients experience it as their own product; the badge on each item says who made it.

Resolution

At request time the server resolves the effective role, Fair version $\oplus$ company version $\oplus$ personal version, field by field, dropping any change the field's policy does not allow at that layer. Resolution is pure and cached per request, so every page, CeMO turn, deck and cron run reads the same role. Today's constants in `src/roles/model.ts` become Chorus 1.0, Atlas 1.0 and Spark 1.0, and stay as the fallback.

Two things are deliberately **not** in a role: the data itself (brands, handles, relevance terms and topics are set up per workspace by data ops) and numbers (a threshold is a role setting, but computing against it stays in SQL).

The three roles: Chorus, Atlas, Spark

Each role is a Fair product with a codename, a version and an owner in the product team. The codename is what clients see on their team ("You're on Chorus 1.2"); CeMO stays the assistant's name on every role.

Codename	Role	Why the name
Chorus	PR	Reputation is a chorus: many voices about you, and the one voice you answer with.

Codename	Role	Why the name
Atlas	Brand & KOL	The map of the whole category: competitors, creators and campaigns, and where you stand among them.
Spark	Social Media	What sets content off: your own posts, formats, timing and the community that answers.

Versions read **major.minor**. A minor release adds or fixes skills, deck templates, tiles or wording (Chorus 1.0 → 1.1). A major release changes how the role behaves: its dashboard layout, its signals, CeMO's voice (Chorus 1.x → 2.0). Clients that follow the latest get minor releases quietly and major releases after a preview banner.

What every role carries

Every role has the same ten parts, so each role owner knows exactly what they own and the Role Lab knows what it may change. Today a role is fifteen fields in

`src/roles/model.ts` ; v2 organises them into these parts, the last of which is new.

Part	What it decides	Who may change it
Identity	Codename, version, description, colour, icon in the team picker and sidebar	Fair only
Navigation	Home page and sidebar entries, from the fixed set Dashboard, Weekly Reports, Decks, Pulse, Chats, Reports	Fair; the Builder may hide or reorder
Dashboard	Which tiles show, their order, default window, brand in focus and platform	Builder; window, focus and order also per Member
Signals	Thresholds that move a status or flag a post: Chorus's ladder (watch / issue / crisis multiples, comment floor, baseline and recovery days), Spark's watch rules (underperform %, comment-storm floor, minimum posts per account)	Shape: Fair only. Numbers: Builder, within guard rails
Skills	The analyses the role offers, their order in the composer, plain titles and example questions	Fair skills: Fair. Company skills: Builder (Members propose)

Part	What it decides	Who may change it
Decks	Templates on offer, default slides, grain, cadence, cover wording	Fair templates: Fair. Company templates: Builder (Members propose)
CeMO	Voice on this team, house rules, what it drafts, what it redirects, empty-screen title and intro, suggested questions	Voice: Fair only. House rules and suggestions: Builder. Answer length: each Member
Vocabulary	Words the product uses ("issue" or "isu", "creator" or "KOL", "own accounts") and the client's name	Builder
Delivery	Scheduled reports and alerts, who gets them, quiet hours	Builder; own inbox per Member
Tests	Golden questions, screens, draft rubrics and guards a version must pass before release	Fair only; problems clients report become new tests

Each part is a typed block in the role's spec (see Technical design). Guard rails belong to the Fair role: a Builder may set Chorus's crisis multiple between 2× and 6× but not below the issue multiple, and the comment floor cannot drop under 20, because below that a status is noise.

The **test set** is what makes fast improvement safe. For each role it holds:

- **Golden questions** — 30 to 50 real questions per role ("Is anything building into an issue for GoPay?"), each with what a good answer must do: which analysis answers it, every number with its evidence, no skill or tool name in the thread, at most one clarifying question.
- **Screens** — the role's dashboard and decks rendered on the test workspaces (Fintech, Beauty, Maudy), checked for overflow, empty tiles and wording that does not fit the category.
- **Drafts** — holding statements, captions and briefs, scored against a short rubric: uses only what the data showed, confirmable facts in [brackets], no invented promise.
- **Guards** — rules no version may break: numbers only from SQL, **relevant is not false** respected, nothing read from another workspace.

A version is released only when its test set passes. Every problem a client reports becomes a new test, so the set grows with use.

The three roles today

Chorus 1.0 — PR (coral). Home is the reputation dashboard: status ladder, share of voice, sentiment and net sentiment by brand, issues labelled *only us / whole category / mixed*, amplifiers, service complaints kept apart from reputation stories. Signals: watch at 1.5× the 28-day negative baseline, issue at 2×, crisis at 3× with at least 50 negative comments, a 50-comment floor, recovering within 7 days. Decks: reputation weekly, reputation monthly, issue post-mortem. CeMO is the comms advisor: drafts holding statements, spokesperson Q&A, replies and escalation notes with confirmable facts in [brackets], sends service complaints to customer care, never advises joining a pile-on.

Atlas 1.0 — Brand & KOL (blue). Home is the competitor dashboard: scoreboard by platform, the landscape (tiers, products, posting, close-ups), creators, campaigns. It covers every platform the workspace holds. Decks: the competitor templates and the Weekly Competitor Pulse. CeMO is the category analyst; its voice is empty today and inherits the workspace persona, so a written voice is the first thing its owner should ship (Atlas 1.1).

Spark 1.0 — Social Media (mint). Home is the own-accounts dashboard: posts per account, engagement as the measure, views only where reported, formats, posting times, community replies, own accounts against competitors. Signals: a post underperforms at 50% below its account's median, a comment storm at 20 negative, an account needs 5 measured posts before it is judged. Decks: content monthly, content teardown. CeMO is the content lead: drafts captions, calendars, replies and briefs.

Example: GoPay's Chorus

After a month of use, GoPay's company version of Chorus 1.0 on `fintech-id` might hold:

- Crisis at 4× with at least 100 negative comments (set by the Builder, within guard rails), because fintech baselines are louder than beauty's.
- The Amplifiers tile hidden and Service complaints moved to the top.
- House rules: "Never draft anything about OJK or Bank Indonesia without saying Legal must review" and "Refer to the client as GoPay, never Gojek".
- Vocabulary: "isu" for issue.
- A company skill, *Merchant complaints by city*, created by a Member and approved by the Builder, badged GoPay.
- A company deck template, *Board monthly*, saved from the reputation monthly.
- An extension: a Persona table (see Client extensions).

None of that touches Fair's Chorus. When the PR role owner releases Chorus 1.1 with a new *Regulator mentions* tile, GoPay gets it (if it follows the latest) and keeps everything above.

The CMS: Fair's brain

The CMS is where Fair builds and runs the product. Only Fair staff sign in to it; clients never see it. It lives at `/admin` in the same app, behind the same sign-in, and nothing in it is reachable over the MCP connector.

Who works in it

Person	What they do	Can release?
Owners — Refal, Rafli	Everything: release role versions, deploy code, add roles, switch a workspace live, set prices, manage Fair staff	Yes, the only ones
Product team — role owners: Audia, Wega, Arneta	Improve the roles in the Role Lab, curate their test sets, read their signals, review client creations for promotion. For now all three own all three roles; Audia and Arneta may also change design (dashboard layouts, deck and slide designs)	No, they propose. They may roll their role back
Data ops	Set up each client's workspace and keep it healthy: sources, loads, brands and handles, relevance, topics, followers, content links, performance metrics, freshness	No

One person can hold more than one duty: a role owner might also run data ops for one client.

The areas

1. **Home** — what needs attention today: versions waiting for Refal or Rafli, failing health checks, Builder approvals stuck for more than a week, credit balances running low, problems clients reported.
2. **Roles** — one page per role (Chorus, Atlas, Spark): its owner, current version, version history with release notes, which clients are on which version, the test set and its last results, signals and insights.
3. **Role Lab** — the AI workspace for improving a role (next section).

4. **Releases** — proposed versions waiting for release, staged rollouts in progress, rollbacks.
5. **Client creations** — every skill, deck template, extension and house rule clients have made, across workspaces, with how much each is used; the source for promotion into a Fair role.
6. **Workspaces** — every client workspace: kind, status, roles and versions, client brand, Builders, last load, health, credit balance. Creating one starts the wizard (see Workspace lifecycle).
7. **Data** — per workspace: sources, loads and their reports, the daily sync, brands and handles, relevance terms, topics, schema modules, and the client's extensions (read-only).
8. **Health** — freshness per platform, rows per day, relevance rate, share in the catch-all topic, follower captures, content links and performance coverage, unusual bursts, failed crons.
9. **People** — Fair staff and their duties; per client, Builders, Members, invites, last seen.
10. **Credits** — balance and use per workspace, price list, top-ups, and for owners the real cost against credits sold.
11. **Audit** — every change in the CMS and every change made on the client side.

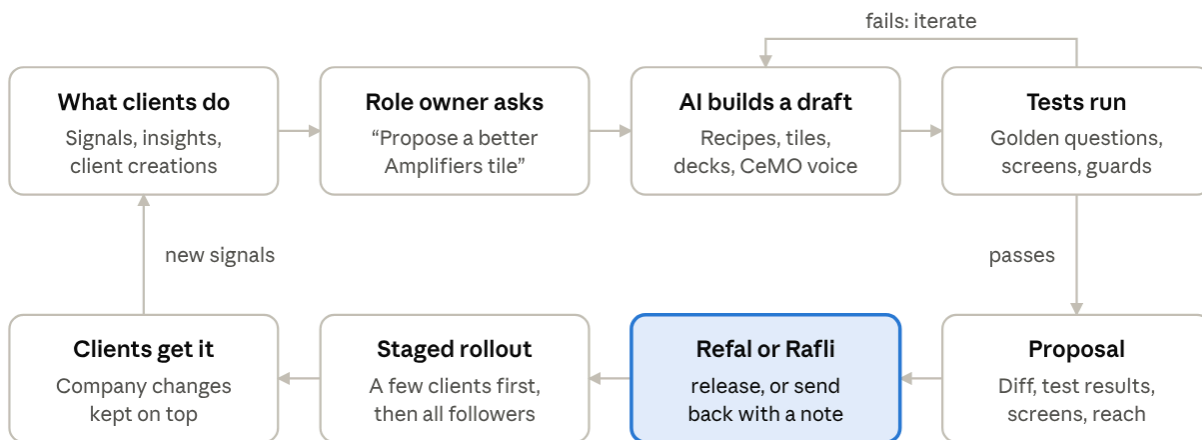
Design principles

- **Test before release, preview before apply.** A role version runs its tests before anyone can release it; a change that alters what is counted shows its before and after on real data first.
- **Plain words.** The product's language, not "entity", "filter predicate" or "tenant".
- **One place per fact.** A brand's handles live in the CMS, not in a JSON contract and a settings blob and a seed CSV. The contracts in `etl/listening/` and `etl/profiles/` become exports of CMS state.
- **Scripts keep working.** `etl/load_listening.py` and `etl/load_profile.py` stay callable by engineers; the CMS calls the same steps.

Role Lab: improving a role with AI

The Role Lab is where a role owner improves their role with AI, the way Refal improved PR, Brand & KOL and Social Media with Claude: ask, let the AI build it, test it on real data, look at the result, iterate, then propose a release. It is the product team's main tool, and the reason the product can move fast without a developer in every change.

A role change reaches clients only after its tests pass and a release



The role owner can roll any release back in one click; clients return to the previous version

Role Lab · from a signal to a release

The AI does the building and the tests do the checking; people decide at two points, when the role owner proposes and when Refal or Rafli release.

A session

Each Lab session works on a **draft version** of one role (for example "Chorus 1.1 draft"), never on what clients use. The role owner talks to the Lab's AI in a chat on one side; the other side shows the draft as it changes: the dashboard on a test workspace, a deck, CeMO's answer to a golden question, the diff in plain words.

What a role owner might ask:

- "PR teams keep hiding Amplifiers. Look at the signals and propose a better tile."
- "Add a skill that lists merchant complaints by city, with three golden questions for it."
- "Make CeMO's holding statements shorter; the closing line is the part people edit most."
- "Build a Board monthly deck template from the reputation monthly, without the amplifier slides."
- "Three fintech clients made their own Regulator mentions skill. Draft a Fair version of it."

What the Lab's AI can do, in the draft only:

- read the role's spec, its signals and insights, and client creations;
- change any part of the draft: tiles, skill recipes, deck templates, CeMO's voice and suggestions, signals within their shape, vocabulary;
- write new skills as **recipes** on the query builder, never as SQL;

- add tests to the test set, and run it;
- render the dashboard and decks on the test workspaces and show the screenshots;
- write the release note.

What it cannot do: release, change design unless the session belongs to a role owner with design rights (Audia, Arnet), change another role, read or change client data beyond the test workspaces, change the core tables, or change code. A change that needs code (a new chart type, a new kind of analysis recipes cannot express) becomes a **code request**: the Lab writes it up with the spec and the tests it should pass, and it goes through Claude Code, a pull request and a deploy by Refal or Rafli, as today.

Skills as recipes

Today every skill is TypeScript in `src/skills/`. For the Lab to be fast, most skills become **recipes**: a declarative spec over the whitelisted query builder. A recipe names the dimensions, measures, filters and windows it asks for; how the result is shown (table, ranked list, chart); the evidence it returns; its plain title, activity line and example questions; and the roles and data modules it needs. The runner executes a recipe exactly like a skill: numbers in SQL, evidence required, "unavailable" when a module is off. Hand-written skills stay for what recipes cannot express and register the same way. That keeps "the model never computes numbers" and "no raw SQL from model input" true for everything the Lab, a Builder or a Member creates.

Tests decide, owners release

A draft cannot be proposed until its test set passes. The proposal is one screen: the release note, the diff in plain words, the test results against the current version (better, same or worse per question), screenshots, and the reach (which clients follow the latest, which have changed the parts it touches). Refal or Rafli read that screen and either release, release to a few clients first, or send it back with a note.

The Lab runs on the same model as CeMO, on Fair's account, with its use logged per role owner. While drafting it runs only the tests a change touches; before a proposal it runs the full set.

Releases, rollout and rollback

- **Release** — only Refal or Rafli. A release creates a new version (Chorus 1.1) with its release note and test results attached.
- **Staged rollout** — first to chosen clients (or Fair's own internal workspaces), then, after a set number of days, to every client that follows the latest.

- **Everyone follows the latest** — every client follows the latest release (decided 4 Oct). Pinning stays possible for a contract that needs it, set by Refal or Rafli.
- **What clients see** — a short "What's new in Chorus 1.1" note; a major release shows a 7-day preview banner ("Review · Stay on 1.x for now").
- **Client work is never touched** — a release never removes or rewrites what a client built. Their deck templates (which they version themselves), skills, reports, house rules and extensions stay exactly as they are, and their changes to Builder-editable settings are re-applied on top. If a release retires a Fair item that a client's creation depends on (a slide in their deck template, a measure in their skill), that creation keeps the Fair version it was built on until the Builder chooses to move it.
- **Rollback** — the role owner, Refal or Rafli can roll a version back in one click at any time. Clients on it return to the previous version with their company changes intact. A rollback needs no approval because it only returns to something already released; it is logged and the owners are notified.
- **Outcome** — two to four weeks after a release, the role page compares signals before and after for clients that took it against clients that stayed pinned.

Compatibility

A role's parts refer to things that live in code: tile ids, deck template ids, slide ids, nav keys, hand-written skills. The Lab and the client side only offer ids the running code declares (from registries built at deploy, the way `skills.registry.json` drives the slash menu), and the resolver drops an id the code no longer knows, logging it. Each version records the minimum app version it needs, and cannot be released before that code is deployed.

The client side: Builders and Members

Clients never see the CMS. Everything they shape, they shape inside the product, and most of it by asking CeMO. That is what makes it feel like their own product.

The Builder

One or more per client (a head of PR, a digital lead). The Builder shapes the company version of each role the company uses:

- **By asking CeMO in Builder mode.** "Hide Amplifiers for everyone." "Raise the crisis level to 4×." "Make a skill that shows complaints by merchant city." "Add a Persona table from our customer research." CeMO drafts the change, previews it on the company's own data, and the Builder applies it. Builder mode is a switch in Chats that only Builders see.

- **Directly where they work.** Customise on the dashboard (hide, move, rename tiles), "Why this level?" on the status ladder with a threshold slider and a 90-day replay, "Save as template" on a deck, the CeMO rules and memory panel.
- **Approving Members' creations.** An approvals inbox: each item shows what it is, who made it, a preview on real data and, if it runs on credits, its cost. Approve, send back with a note, or reject.
- **Seeing the company version whole.** An "Our Chorus" page lists every company change with who made it and when, which Fair version the company is on, what the next Fair release would change for them, and Undo with full history.
- **Suggesting to Fair.** "Suggest to Fair" on any company item sends it to the CMS's Client creations with a note.

Members

Members use the role every day: Chats, Dashboard, Decks. They make it their own:

- Personal settings apply at once: default window, brand in focus, tile order, which alerts reach them, answer length.
- They **create** skills, deck templates and report formats by asking CeMO ("Make me a weekly report of GoPay complaints by product"). A creation works for its maker in preview and goes to the Builder for approval before it joins the company version, appears for everyone and runs on a schedule.
- They propose house rules and vocabulary the same way.
- They see the status of what they made: draft, waiting, approved, sent back.

Badges

Every skill, deck template, tile, house rule and extension shows who made it: **Fair** for what came with the role, the client's name (and the maker, on hover) for what the company made. Clients see clearly what they built; Fair sees clearly where everything came from.

What client changes may and may not do

- **May:** change Builder-editable parts within guard rails; create company skills as recipes (same builder, same rules as the Lab), company deck templates from the slide library, report formats, house rules, vocabulary and data extensions.
- **May not:** change locked parts, the core tables, how a number is computed, or anything in another client's workspace.
- **Limits per plan:** how many company skills, deck templates and extensions a workspace may hold, and its credit pool, are set per workspace in the CMS.

CeMO memory and house rules

House rules ("Never draft anything about OJK without saying Legal must review") and memory facts ("our fiscal month starts on the 26th", "Paylater launches 1 Nov") live in the company version. When someone pushes back in a way that should last ("We call it isu, not issue"), CeMO answers in one plain sentence and offers "Remember this for the team?"; a Member's offer goes to the Builder. Corrections about data ("Don't count Shopee as a competitor here") go to Fair data ops instead, because a prompt cannot change a number.

Every change, recorded

- Each change is saved with who, when, layer, part, old value and new value.
- Each change can be undone: a "Changes" list on each surface shows the last ten, and "Our Chorus" shows all.
- When a company change hides something the Fair role shows, the surface says so in small text ("Amplifiers hidden for GoPay · Show").
- Each change emits a signal for the learning loop, whether it was kept, approved, sent back or undone.

Client extensions: their own tables and metrics

Clients can enhance their dataset without changing Fair's. The core tables (posts, comments, snapshots, creators, brands, topics) are the same for every client; an extension adds to them, for one workspace only.

What an extension can be

- **A custom field** on posts, comments or creators: *Persona* on a creator, *Product line* on a post, *Complaint severity* on a comment.
- **A custom table** linked to core rows: *Personas* (name, description, how to recognise one), *Campaign calendar* (campaign, dates, budget), *Merchant list*.
- **A custom metric**: a formula over core and custom fields, computed in SQL, such as "share of complaints that mention fees" or "engagement per 1K followers by persona".

Where the values come from

Source	Example	Cost
The client's file	A campaign calendar or persona list uploaded as CSV or a sheet	Small

Source	Example	Cost
A rule	Persona = Mom when the creator's bio mentions "ibu" or "mama"	Small; computed in SQL
CeMO reading each row	CeMO reads each creator's bio and captions and assigns a persona	Per row, and again for every new row: the main cost

How a Builder makes one

The Builder describes it to CeMO in Builder mode: "Add a Persona table: Mom, Student, Office worker, Beauty enthusiast. Tag every creator from their bio and captions." CeMO drafts the definition, tags 50 rows as a sample for the Builder to check, and shows an estimate ("Tagging 4,210 creators uses about 2,100 credits, then about 60 credits a day for new creators"). The Builder approves; a job fills it; the extension then appears in the query builder, the dashboard, skills and decks like any other dimension, with the client's badge. A Member can propose an extension too; it waits for the Builder like any other creation.

Use case: two beauty clients

Paragon works on the standard beauty dataset and needs nothing extra. A second beauty company has its own persona model and its own definition of a "hero product". Both run Atlas on the same core tables; the second adds a Persona table and a Hero product field, and its company skills and decks use them. Paragon sees none of it, and Fair's core does not change.

From extension to core

Fair reviews extensions in the CMS's Client creations. When several clients build similar ones (personas keep appearing in beauty), the role owner can design a Fair version through the Role Lab, plus a code change if it needs new core columns, and Refal or Rafli release it. Clients who built their own keep theirs and can switch to the core one when they choose.

Rules

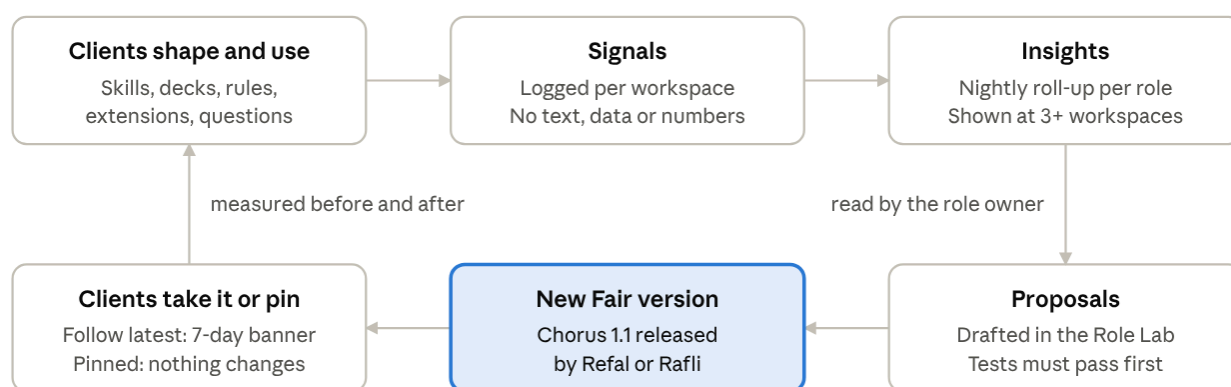
- No database changes from model input: an extension is rows in definition and value tables (see Technical design), read by the query builder. The model names values; SQL counts them.
- Extension data belongs to the workspace and is deleted with it.

- CeMO reading rows always shows a sample and an estimate first, needs the Builder's approval, and stops at the workspace's credit cap.

The learning loop: how roles get better the more teams use them

This is what makes each role better over time. Every client shapes its own version; Fair sees, across all of them, what people make, approve, keep, change, use and ignore, and folds the patterns that hold into the next release. The loop has five steps, and it runs at three levels.

Each role gets better as more teams use and reshape it



learning loop · 6 steps

Only settings and counts travel around this loop; data and text stay in their workspace (the boundary is stated at the end of this section).

Three levels

The same loop runs inside each client and across clients:

- **Member → Builder.** A Member makes a skill, deck template or report; the Builder approves it into the company version.
- **Builder → Fair.** Company items and how much they are used appear in the CMS's Client creations; the role owner can bring a pattern into the Fair role through the Role Lab.
- **Fair → every client.** A release reaches every client that follows the latest.

Each step is recorded, so a release can say where it came from ("started from creations in three fintech workspaces").

1. Signals

A signal is one small, structured event, written to `model_events` with workspace, role, person, model version, surface, kind and a small payload. There are two families.

Shaping signals come from the client side: a creation made, approved, sent back or rejected; a tile hidden; a threshold moved; a template saved; a house rule added; an extension built; a suggestion replaced; a skill hidden; and every Undo of those.

Use signals come from normal work:

Surface	Signals
Dashboard	tile viewed (in viewport ≥ 2 s), "Ask why" clicked, filter changed, time on page
Signals	status changed, alert sent, alert opened, "not an issue" pressed, ladder drawer opened
Decks	template chosen, slide removed or reordered, wording edited (which slide, not the text), exported, emailed, version made by cron and opened or not
Chats	analysis run (skill and layer), clarifying question asked, pushback, "Show chart", evidence pane opened, export, "Turn into a report / deck", answer copied, conversation abandoned after one turn
CeMO drafts	draft requested (holding line, Q&A, caption), kept, edited heavily, discarded

What a signal never carries: the text of a question or answer, a comment, a post, a brand name from another workspace, or a number from the data. Questions are reduced to the *analysis that answered them* and a coarse intent label computed inside the workspace ("issue check", "competitor comparison", "draft"). This is the privacy boundary, and it is what lets learning cross workspaces without anything else crossing.

2. Insights

A nightly job (Vercel Cron) rolls signals up per role and version into `model_insights`. Each insight is one sentence with its counts and the workspaces behind it, for example:

- "Crisis multiple raised from 3× in 6 of 9 PR workspaces; median chosen value 4×."
- "Amplifiers tile hidden by 5 of 9 PR workspaces within two weeks; viewed under 2 s in the others."
- "Brand & KOL: 41% of conversations end in a Top content run; Top content is 6th in the composer."
- "Social: content-teardown deck is made and never opened in 70% of monthly runs."

- "PR drafts: holding statements kept 3 in 10 times; the most edited part is the closing line."

Insights only count; a person on the product team reads them. A minimum of three workspaces is required before an insight about a setting is shown, so no single client's choices are visible as such.

3. Proposals

The role owner takes an insight into the **Role Lab** and turns it into a draft version: a diff against the current release ("Chorus 1.1: crisis multiple 3× → 4×; Amplifiers tile off by default; Service complaints moved up"). Before it can be proposed, the draft passes the role's test set, and the proposal shows its reach: how many clients follow the latest, how many have already changed each part (and so will not feel it), and for any number-changing part, a replay of the last 90 days per client under the new value ("GoPay: 4 days fewer at crisis"). The replay is SQL over history, never an estimate.

Proposals also start from clients. A Builder can press "Suggest to Fair" on any company item, and every client creation is visible to its role owner in the CMS, with the client named only to Fair staff.

4. Versions and rollout

Refal or Rafli release a proposal as a new version. Rollout respects each client's follow choice:

- **Follow latest:** the workspace sees a banner for 7 days ("Chorus 1.1: Service complaints moved up, crisis now 4×. Review · Keep old"), then the new version applies. Its company changes are re-applied on top; one that no longer fits the new version is kept aside and shown to the Builder, never silently dropped.
- **Pinned:** nothing changes. The Builder's "Our Chorus" page and the CMS show "2 versions behind" and what each version would change.

The role owner, Refal or Rafli can roll a version back at any time; clients on it return to the previous version with their company changes intact.

5. Measure

Each version records the signals that motivated it. Two to four weeks after rollout, the role's page in the CMS compares the same signals before and after for workspaces that took the version, against workspaces that stayed pinned (a natural control). "After Chorus 1.1, alerts opened rose from 31% to 54%; pinned workspaces unchanged." That closes the loop and is the evidence for the next proposal.

The boundary, stated once

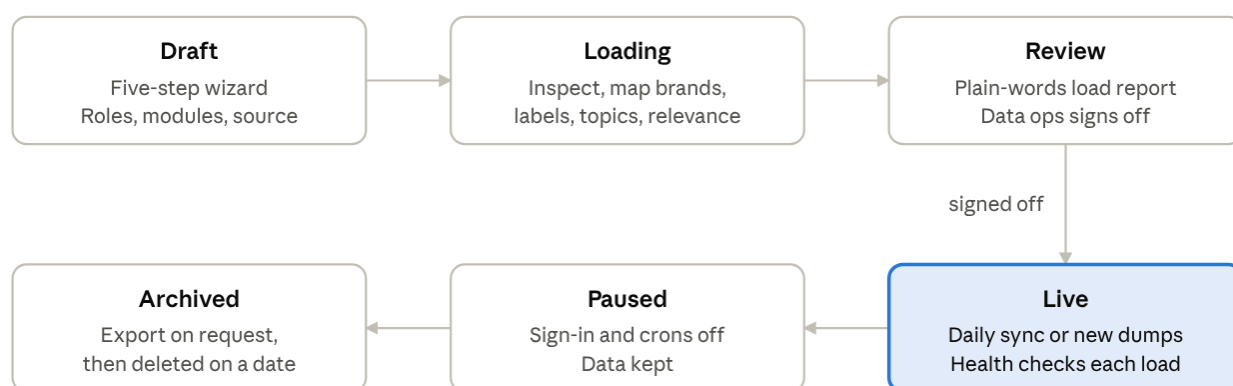
Across workspaces moves only: which parts were changed, to which setting values (thresholds, tile ids, template ids, recipe shapes), and counts of use. Never: data rows, extension values, text people typed, CeMO's answers, house-rule wording, memory facts, brand names, or numbers computed from a workspace's data. House-rule *wording* stays in its workspace; the CMS's roll-ups see only that a rule was added to the "drafting" or "naming" category. Client creations are visible to the role owner one by one (that is how promotion works), but appear in cross-client insights only above the three-workspace floor. A client's contract can switch its workspace out of learning entirely (`settings.learning = false`); its signals and creations then stay visible to Fair only on that workspace's own page.

Workspace lifecycle

Onboarding Fintech took one engineer most of a day: reading the dump, asking you how to group brands and map sentiment, writing `etl/listening/fintech-id.json`, running the loader, finding that 27% of posts did not name their brand, adding the relevance flag, checking screens. Each of those steps becomes a screen in the CMS, so data ops can onboard a workspace in an hour and the next workspace (a bank, a telco, a beauty brand on listening) needs no code.

A workspace moves through six states: **draft** → **loading** → **review** → **live** → **paused** → **archived**.

A workspace goes live only after a person signs off its load report



workspace lifecycle · 6 states

Clients only ever see a live workspace; review is where a person reads the load report before anyone else does.

1. Create (draft)

A five-step wizard:

1. **Basics** — name, id (slug, fixed once live), kind (category panel, profile, listening), category (beauty, fintech, telco, FMCG, other; drives which lexicons and schema modules apply), language, time zone (Asia/Jakarta default).
2. **Client** — the client brand, if any (today `workspaces.client_brand_id`), and the client's display name.
3. **Roles** — which teams the workspace offers (PR, Brand & KOL, Social Media) and, for each, which version (Chorus, Atlas, Spark) it starts on and whether it follows latest. Defaults by kind: profile → PR, panel → Brand & KOL, listening → all three.
4. **Modules** — core listening always; beauty commerce, creator tiers and caption reading switched on by category.
5. **Source** — upload a dump (zip) now, or connect the daily sync once it exists.

2. Load (loading)

This step makes the **default dataset**: what Fair Listening collects for this client, mapped onto the core tables. Anything the client wants beyond it comes later from their side, as extensions (see Client extensions). The dump is stored, then read by a job. The loader is today's Python, split into steps the CMS can show and stop between:

- **Inspect** — files found, platforms, date range, row counts, columns recognised and unrecognised. For Fintech: 13,275 posts, 116,366 comments, 114,633 snapshots, X/Threads/TikTok/Instagram/YouTube.
- **Map brands** — handles found in the dump, grouped by suggestion (same name across platforms, known aliases), for data ops to confirm, merge ("BCA + blu by BCA") or drop. Owned accounts ticked here.
- **Map labels** — the dump's sentiment scale to the product's three classes ("Average → negative" for Fintech), with counts per mapping so the effect is visible.
- **Topics** — the dump's topics imported as the workspace taxonomy, with share per topic; anything over 30% in a catch-all is flagged.
- **Relevance** — terms proposed from brand names and handles, with the share of posts that would count and examples on each side (see Brands, relevance, topics and schema modules).
- **Write** — rows inserted with `workspace_id`, views refreshed.

3. Review

A **load report** in plain words, the same one kept for every later load:

- Totals by platform and day, with thin days flagged (the last days of a dump are often incomplete; the dashboards already end windows on the last settled day).
- Relevance: 3,548 of 13,275 posts (27%) do not name their brand; Threads is about half off-topic.
- Topics: "Others" holds 42–59% of comments; Paylater has 10 comments.
- Bursts and gaps: own Instagram posts for @gopayindonesia jump to 57–65 a day on 7–9 Sep and stop after 9 Sep.
- Missing data: follower history has one capture per account, so follower growth is shown as unavailable.
- A screenshot strip of each team's dashboard on this data, so data ops sees what the client will see.

Data ops signs off, Refal or Rafli switch the workspace live, and the invite to the client's first Builder goes out. Problems become **notes** on the workspace that CeMO can read ("Own Instagram posts are not captured after 9 Sep"), so it does not draw conclusions from a gap.

4. Live

- **Daily sync** (when the team's read-only sync exists) runs as a scheduled load; each run gets a load report. Until then, new dumps are uploaded and loaded as increments, de-duplicated on (platform, url, brand).
- **Health** checks run after each load and nightly: freshness per platform, followers captured, content links and performance metrics present, rows per day vs the 28-day usual, relevance rate, catch-all share, unlabelled comments backlog, crons that failed. A check that fails twice raises a note to data ops and, if it affects numbers on screen, a quiet banner in the product ("TikTok data is 3 days behind").
- **Re-load and roll back** — every load is a batch with an id; a bad load can be removed and the views refreshed.

5. Paused

Sign-in disabled for client members, crons and scheduled decks stopped, data kept. Used between contracts.

6. Archived

Data exported to a file for the client on request, then deleted on a set date; its company versions, creations, extensions and memory are deleted with it. Its anonymous signals remain in the learning roll-ups, because they no longer identify it.

Brands, relevance, topics and schema modules

These are workspace **data settings**, set by Fair data ops in the CMS, not role settings: they decide what is counted, so every role on the workspace sees the same result. Changing one is a number-changing edit and goes through preview and approval (see Governance).

Brands

- **Brand set** — canonical slug, display name, nickname CeMO may use, logo, colour, whether it is the client, whether it is shown on dashboards (a brand can be loaded but kept off the competitor set).
- **Handles** — per platform, many per brand; each marked owned or not. Owned vs earned then works on every platform the workspace holds, as it already does for listening workspaces.
- **Groups** — merges like "BCA + blu by BCA" or "Shopee + ShopeePay" (kept separate in Fintech today). A merge is reversible because it maps handles, not rows.
- **Competitor sets** — named subsets ("E-wallets", "Banks") that teams can pick as a filter and that decks can take as their brand list.

Relevance

Today a listening post counts when it is from an owned account, tags the brand, or its caption matches one of the brand's terms (a term matches at the start of a word; an all-caps term is case-sensitive and must be a whole word). That rule is right for any workspace; what differs is the terms. The CMS makes them a tool:

- **Terms editor** per brand, with three lists: *counts* ("gopay", "GoPay Later"), *only with* ("OVO" counts only as a whole capitalised word), *never* ("go pay attention").
- **Live preview** — as terms change, the panel shows the share of the brand's posts that would count, by platform, with ten examples flipping in each direction and their views. Fintech today: Threads is about 50% off-topic; X mostly on-topic.
- **Apply** recomputes `posts.relevant` for that brand in a job and refreshes the views. The before/after (posts, views, share of voice) is stored with the change.
- **Suggestions** — handles and words that often co-occur with the brand's tagged posts are offered as candidate terms; a person always confirms.

This answers your first decision from the PR plan: off-topic filtering works for any onboarded workspace, because the rule is general and the terms are workspace data edited here, not code.

Topics

Topics arrive with the dump today (Fintech: Others, Promo, Service, Paylater and more, with "Others" holding 42–59% of comments and Paylater only 10). You said topics are adjustable per workspace; the CMS makes that concrete:

- **Taxonomy editor** — rename, merge, split, reorder (`topics.sort_order`), mark the catch-all (`is_catch_all`), give each topic a one-line definition and three example comments.
- **Health** — share per topic over time, with a warning when the catch-all passes 30% or a topic falls under 30 comments a month.
- **Mapping** — when the listening team changes a topic upstream, a mapping table keeps the workspace's names stable.
- **Role use** — each topic can be tagged *service, promo, product, reputation*; PR's service list and "only us / whole category" logic read those tags instead of the theme matching in code today.

Re-labelling the catch-all with our own model is possible later (`/api/cron/label` already labels profile comments) but is a separate decision, listed under Decisions.

Schema modules

The complete listening schema is the template for every workspace (DECISIONS, 3 Oct 2026). On top of it, optional modules switch on extra data and the screens that use it:

Module	Adds	Used by
Core listening (always)	posts, comments with sentiment, topics, snapshots day 0–30, owned vs earned, relevance	all roles
Beauty commerce	product type, content format, affiliate signals, product lexicon	Brand & KOL dashboards, landscape, products
Creator tiers	follower bands from <code>src/config</code> per category	Brand & KOL, Social comparisons
Caption reading	product, event, offer, hook, angle per post (<code>posts.cap_*</code>), floor and start date	decks, query builder
Comment labelling	our own three-class sentiment where the source has none	profile workspaces

Each module declares the columns it needs, its config (bands, floors), and which skills and slides depend on it. A skill whose module is off returns "unavailable" with a plain message, as today. Tier and affiliate bands stay in `src/config` as defaults and become per-category overrides in the CMS, still read by SQL, never hardcoded. Modules are Fair's and switched on by data ops; what a client adds on top is an extension.

People and access

Five kinds of people

Who	Side	Signs in to	Can
Owner — Refal, Rafli	Fair	CMS, and the product on any workspace	Everything. The only ones who release role versions, deploy code, switch a workspace live and set prices
Product team — role owners	Fair	CMS; the product on test workspaces	Improve their role in the Role Lab, propose releases, roll back their role, review client creations
Data ops	Fair	CMS; the product on their workspaces, read-only	Set up and run workspaces: sources, loads, brands, relevance, topics, health
Builder	Client	The product, their workspace	Shape the company version (Builder mode), approve Members' creations, build extensions, invite and remove Members, see credit use
Member	Client	The product, their workspace	Use the role every day, personal settings, create skills, decks and reports for the Builder's approval

A client can have several Builders. A person can be a Builder on one role and a Member on another in the same workspace: GoPay's head of PR builds Chorus and only uses Atlas.

Accounts and memberships

Today a person is a row in `users` per workspace (unique on workspace + email), with `role` owner or member; owners may switch workspace, and the team is a cookie (`fi_role`) any member can change among the workspace's roles. v2 splits identity from access:

- **Accounts** — one per person (email, name, password hash or SSO link, WhatsApp). One sign-in reaches everything the person belongs to.
- **Memberships** — account × workspace, with Builder or Member per role the person may use there.
- **Fair staff** — a flag on the account plus duties (owner; role owner of Chorus; data ops for fintech-id). Every time Fair staff open a client workspace, it is logged in the audit log.

Every page, route, CMS handler, Builder-mode tool and MCP call checks one server function, `can(account, workspace, role, action)`.

Teams and sign-in

- A person with one membership and one role lands on that team's Dashboard. With more, they choose on `/persona`, which lists the workspace × role pairs they hold.
- The sidebar's workspace picker and role tabs show only what the membership allows; `fi_role` stays as the remembered choice, and the server rejects a role the membership does not grant.
- The MCP connector keeps scoping by its token's workspace; tokens gain the membership's role, so CeMO's voice over MCP matches the team.

Invites

Data ops invites the client's first Builder at go-live; Builders invite Members. Invites go by email through Resend, expire in 7 days, and can be sent in bulk from a CSV. Removing a membership signs the person out of that workspace at their next request.

Migration

Existing `users` rows become accounts (merged by email) and memberships: an owner who is Fair staff becomes a Fair owner or data ops, any other owner becomes a Builder, and a member stays a Member. No one has to reset a password.

Credits and billing

Dashboards, filters, brand pages, approvals and settings cost Fair nothing per click: they are SQL. Model calls cost money: CeMO's answers and drafts, deck writing, Builder-mode changes, Member creations, extensions that CeMO fills, and the caption and comment readers. v2 bills those in **credits**, a unit Fair prices, never in dollars.

What costs credits

Action	Credits (placeholder)
A question to CeMO	1
A draft: holding statement, caption, brief	2
A deck or report version	5
A Builder-mode change, drafted and previewed by CeMO	3
A Member creation: skill, deck template, report format	3 to create, then normal use
An extension that CeMO fills	Per 1,000 rows read, always shown before it runs
Dashboards, filters, settings, approvals, exports	0

The numbers are placeholders until a month of real cost per action is measured. Tokens are already recorded per chat message; phase 1 records them for decks, crons and every other model call too, per action and per workspace.

How it is sold

- Each workspace's plan includes a monthly credit pool shared by its people, with top-ups at any time. Plans also set the limits on company skills, deck templates and extensions.
- A credit's price comes from the measured average cost per action plus Fair's margin, which also covers crons, the database, retries and support. Clients see credits, never a dollar figure presented as cost.
- Builders see use by person, kind and day, get an alert at 80% of the pool, and can set a monthly cap. At the cap, actions that use credits stop; dashboards keep working.
- Heavy actions (an extension read by CeMO, a large backfill) show an estimate and need the Builder's approval.

- The CMS's Credits area shows Refal and Rafli the real cost against credits sold per workspace.

Other ways to pay, later

- **Their own API key.** An enterprise client with an Anthropic API account pastes a key in the CMS; CeMO and Builder mode run on their bill, and Fair charges the plan only.
- **Their own Claude, through the connector.** Power users can connect Fair Intelligence to their own Claude through `/api/mcp`. Their subscription pays for the model and Fair's server only runs the SQL tools, but they lose the dashboards, decks and CeMO's role voice there. Builder tools over the connector are possible later, with every rule enforced on the server.
- A consumer Claude subscription cannot power CeMO inside the product.

Governance: audit, approvals, recompute, privacy

When clients can reshape the product, two questions come up in every client meeting: *who changed this number*, and *can another client see our setup*. When Fair can release new versions every week, a third comes up inside Fair: *who let this go out*. Governance answers all three.

Kinds of change

Kind	Examples	What it needs
Personal	Window, brand in focus, tile order, inbox	Applies at once, undoable
Company presentation	Tiles shown, vocabulary, suggestions	The Builder applies it; undoable
Company creation	Skills, deck templates, report formats, house rules	Made by a Member or the Builder; live after the Builder approves
Company number-changing	Thresholds, extensions	The Builder approves after a preview on real data; an extension CeMO fills shows a sample and an estimate first
Data number-changing	Relevance terms, brand groups, handles, owned flags, topic merges, sentiment mapping	Fair data ops: preview, recompute job, before and after stored

Kind	Examples	What it needs
Fair role version	Anything in Chorus, Atlas or Spark	Tests pass in the Role Lab; Refal or Rafli release; the role owner may roll back
Code	New engines, new core columns	Pull request; deploy by Refal or Rafli

Thresholds sit in the third row even though they only change a status, because a status is a claim the client may act on ("we were in crisis on 14 Sep"). Changing a threshold never rewrites the past silently: dashboards show history under the rule in force at the time, and the ladder drawer can replay under the new rule on request.

Audit

`audit_log` records every change from the CMS and every change made on the client side: who, when, workspace, area, field path, old and new value, reason note, and for number-changing edits, the before/after summary and the job that applied it. It is append-only, readable by Builders for their workspace and by Fair for all, and exportable to CSV. Report and deck versions record the role version, the company version and data load ids they were built on, so a number in an old deck can be traced to the exact rules that produced it.

Recompute

Number-changing edits run as **jobs** (see Technical design): set `posts.relevant` for a brand, regroup handles, remap sentiment, rename topics, refresh materialized views. A job shows progress in the CMS, can be cancelled before it commits, and writes in a single transaction per brand so a half-applied change never reaches a screen. While it runs, the affected dashboards show "Updating counts for GoPay...".

AI safety: CeMO, Builder mode and the Role Lab

House rules shape tone, naming and what to draft. They cannot override the product's own rules, and the server enforces that rather than trusting the prompt:

- House rules are placed in the system prompt after, and subordinate to, the fixed rules (numbers only from tools, evidence for every number, no skill names in the thread, one clarifying question at most).
- A rule that asks CeMO to compute, invent, hide evidence, or speak as a real person outside drafting is refused at save with a plain reason.

- Rules have a length limit and a count limit per team; the effective prompt size is shown on the company version page and in the CMS.

Builder mode and the Role Lab follow the same rule: the AI proposes, the server enforces. Builder mode's CeMO changes only the company version, only parts the policies allow, and every change is previewed before the Builder applies it. The Role Lab's AI changes only the draft of one role, writes skills only as recipes, and cannot release. Neither can write SQL, change a core table or reach another workspace, whatever it is asked.

Privacy

- Every new table carries `workspace_id`; nothing crosses workspaces except the anonymous signals of the learning loop, and only above the three-workspace floor.
- `settings.learning = false` keeps a workspace's signals inside it.
- Memory facts and house rules are workspace data: deleted with the workspace, never in roll-ups.
- Fair staff access to a client workspace is logged in the audit log like any other access.
- No secrets in the CMS's own config; source credentials for the daily sync live in Vercel environment variables, referenced by name.

Technical design

Same stack: Next.js App Router, TypeScript, Drizzle on Neon, Vercel with Vercel Cron, Resend, the Python ETL. No Railway, pg-boss or Redis.

New tables

All carry `workspace_id` except the Fair role tables (versions, Fair recipes, tests) and the account tables, which are global by design.

Table	Key columns	Purpose
<code>role_versions</code>	role, codename, version (major.minor), status (draft / proposed / released / rolled back), spec jsonb, policies jsonb, release_note, min_app_version, proposed_by, released_by, released_at	Fair roles; Chorus, Atlas and Spark 1.0 seeded from <code>src/roles/model.ts</code>

Table	Key columns	Purpose
company_versions	workspace_id, role, version, base_version, follow (latest / pinned), overrides jsonb, author, note	The client's company version, one row per change set; the latest is live
personal_settings	workspace_id, account_id, role, settings jsonb	Member-editable parts only
creations	workspace_id, role, kind (skill / deck_template / report / rule / extension), spec jsonb, maker, status (draft / waiting / approved / sent_back / rejected), approver, note	Everything a client makes, with its approval and badge
recipes	owner (fair or a workspace_id), id, version, spec jsonb	Declarative skills over the query builder, Fair's and clients'
test_cases	role, kind (question / screen / draft / guard), spec jsonb, source (fair / client report)	Each role's test set
test_runs	role_version, case_id, workspace_id, result, score, artifacts	Results per draft, proposal, release and nightly run
lab_sessions	role, draft_version, owner, transcript ref, tokens	Role Lab work and its cost
releases	role_version, stage, workspaces, released_by, rolled_back_by, at	Staged rollout and rollback
ext_defs	workspace_id, kind (field / table / metric), target, name, type, allowed values, source (file / rule / cemo), spec jsonb	Client extension definitions
ext_values	workspace_id, def_id, row_ref, value jsonb, source	Extension values; a custom table is rows keyed by its definition

Table	Key columns	Purpose
workspace_memory	workspace_id, role (nullable), fact, source (taught / Builder), author, created_at	Facts CeMO reads every turn
accounts	id, email, name, password_hash, staff flag, duties	One per person
memberships	account_id, workspace_id, per-role level (Builder / Member), invited_by, status	Replaces per-workspace users rows
invites	workspace_id, email, levels, token hash, expires_at	Email invites
model_events	workspace_id, role, account_id, versions, surface, kind, payload jsonb, at	Signals; payload validated against a whitelist per kind
model_insights	role, version, key, sentence, counts jsonb, workspaces int, day	Nightly roll-ups, nothing named below the floor
credit_ledger	workspace_id, account_id, kind, credits, tokens, cost, ref, at	Every credit spent or added, with Fair's real cost for the margin view
audit_log	workspace_id, account_id, area, path, old jsonb, new jsonb, note, job_id, at	Append-only
cms_jobs	workspace_id, kind, params jsonb, status, progress, error, locked_until	Recompute, load, extension-fill, test and export jobs
data_sources, brand_terms, brand_handles	workspace_id, ...	Where data comes from; relevance and handles as rows

data_loads gains source_id, report jsonb and rolled_back_at; topics gains definition, tags and examples; reports and decks gain model_version.

Effective model resolution

```
// src/roles/resolve.ts
export async function getRoleModel(ws: string, role: RoleId, account?:
string): Promise<Effective> {
  const wm = await liveWorkspaceModel(ws, role);           // cached per
request
  const core = await coreModel(role, wm?.base_version);    // falls back to
ROLES[role] as v1
  const person = account ? await personalModel(ws, account, role) : null;
  return resolve(core, wm?.overrides, person?.overrides); // pure, policy-
checked, drops unknown ids
}
```

Every current use of `ROLES[...]`, `roleNav`, `fillRole`, the role voice in `src/chat/loop.ts`, deck template lists in `src/decks/templates.ts`, the alert numbers in `src/reputation/dashboard.ts` and the watch numbers in `src/social/dashboard.ts` moves to the resolved model. `resolve` stays pure so it can be unit-tested with the same fixtures as `src/roles/__tests__/model.test.ts`. The `spec` shape is a TypeScript type plus a zod schema; `role` versions declare a `schema_version`, and a company change that fails the schema is kept aside, not applied.

Code registries

Tiles, deck templates, slides, nav keys and skills already exist as code. Each gets a small registry (id, label, roles it suits, module it needs) generated at build, the same way `skills.registry.json` drives the slash menu. The Role Lab and Builder mode read registries to offer choices; the resolver uses them to drop unknown ids.

Recipes, Builder mode and the Role Lab

- **Recipes** run through the same runner as hand-written skills (`src/skills/runner.ts`): resolved params, SQL from the query builder, evidence required, "unavailable" when a module is off. A recipe's spec is validated against the builder's whitelist when it is saved, not when it runs.
- **Builder mode** is CeMO's chat loop with a second set of tools, scoped to the company version: propose a change, preview it, draft a recipe, draft a deck template, draft an extension, estimate its cost. Applying is a separate server call that checks policies and the person's level.
- **The Role Lab** is the same loop on Fair's side, with tools scoped to one role's draft: read the role and its signals, edit the draft, write a recipe, add a test, run tests, render screens, write the release note. Renders reuse the Playwright setup used to check dashboards and decks today.

- **Extensions** reach the query builder through `ext_defs` : a custom field becomes a dimension, a custom metric a measure, and the join is generated from the definition, never from model text.
- All three run on the chat model through `toolAnswer` , and every call writes its tokens to `credit_ledger` (Fair's own account for the Lab).

Jobs without pg-boss

`cms_jobs` is a table; a Vercel Cron route (`/api/cron/jobs` , every minute) claims due jobs with `update ... set locked_until = now() + interval '5 min' where id = (select ... for update skip locked)` and runs them in slices that fit a function's time limit, saving progress between slices. Large loads stay in Python: the CMS stores the dump in Vercel Blob and the job runs the Python loader through a Vercel Python function (or, for very large dumps, data ops runs it locally against the same job id). Nothing new to host.

APIs

- `/api/admin/*` — Fair only: workspaces, data, roles, releases, client creations, people, credits, audit; every handler starts with `can()` .
- `/api/lab/*` — Role Lab sessions, draft edits, test runs, renders, proposals.
- `/api/builder/*` — the client side: company changes, creations and approvals, extensions, credit use.
- `/api/model/events` — batched signals from the client, whitelisted kinds only.
- `/api/cron/jobs` (job runner), `/api/cron/insights` (nightly roll-up), `/api/cron/tests` (nightly full test run of every released version on fresh data).

None of these are exposed through `/api/mcp` .

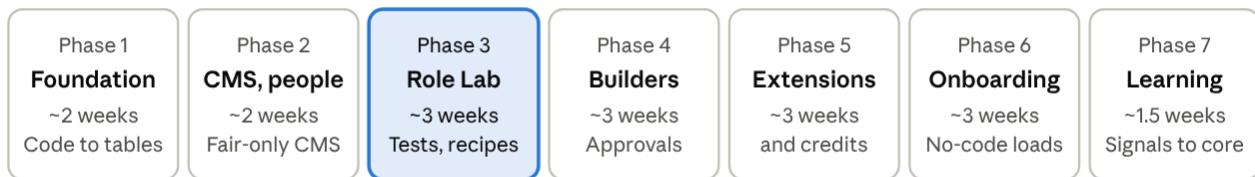
Migrating what is hardcoded today

1. Seed `role_versions` with Chorus 1.0, Atlas 1.0 and Spark 1.0 from the constants; seed `company_versions` for every workspace from `settings.roles` , with no changes.
2. Move `etl/listening/fintech-id.json` and `etl/profiles/*.json` into `brand_handles` , `brand_terms` , the sentiment map and settings; keep writing the JSON as an export.
3. Move `settings.captions` , `settings.roles` , persona copy and `client_brand_id` into CMS forms (the columns stay; the CMS edits them).
4. Turn the most-used skills into recipes first, and check each returns the same rows and evidence as its hand-written version before switching.
5. Switch callers to the resolved role behind a flag; compare it with the constants for every workspace until they match, then remove the flag.

Delivery phases

Seven phases, each shippable on its own and each leaving the product working. Weeks are build time for one engineer with Claude; review and client testing add to them.

Seven phases, about seventeen weeks; the Role Lab is the bet



Each phase ships on its own and leaves the product working

delivery roadmap · 7 phases

The bands are drawn equal; the weeks under each name are the estimate.

Phase 1 — Foundation (about 2 weeks)

Roles move from code into versioned tables with their codenames. Clients see no change except the codename on their team.

- `role_versions`, `company_versions`, `personal_settings`, `audit_log`, `model_events`; Chorus, Atlas and Spark 1.0 seeded from the constants.
- The resolver with policies and guard rails; every caller switched behind a flag.
- Registries for tiles, deck templates, slides and nav keys.
- Releases and rollback as data: a version can be released by Refal or Rafli and rolled back by its role owner (no Lab yet).
- Tokens and cost recorded for every model call, per action and workspace.
- **Done when** every workspace × role resolves to today's behaviour (a test per workspace), and pnpm test, smoke and db:stats pass.

Phase 2 — CMS shell and people (about 2 weeks)

- `/admin` for Fair staff only: Home, Roles, Workspaces, People, Audit.
- Accounts, memberships and invites; the five kinds of people; `can()` everywhere; migration from `users`.
- **Done when** Rafli can see every role's version and every workspace's state in the CMS, and a client Builder can invite a Member while neither can reach `/admin`.

Phase 3 — Role Lab (about 3 weeks)

- Test sets for the three roles (golden questions, screens, draft rubrics, guards), written with the role owners; nightly test runs.
- Recipes on the query builder; the most-used skills converted.
- Lab sessions: chat, draft version, the Lab's tools (edit, recipe, test, render, release note), the proposal screen, release and staged rollout.
- **Done when** a role owner ships Chorus 1.1 (one new recipe skill and one tile change) from the Lab: its tests pass, Refal releases it to Fintech first, then to everyone.

Phase 4 — Builders and Members (about 3 weeks)

- Builder mode in Chats; Customise on the dashboard, the ladder drawer, Save as template, the CeMO rules and memory panel.
- Member creations and the Builder's approvals inbox; Fair and client badges.
- The "Our Chorus" page with history and undo; Suggest to Fair; Client creations in the CMS.
- **Done when** GoPay's Builder makes every change in the GoPay example by asking CeMO, a Member's skill goes through approval, and the CMS shows it under Client creations.

Phase 5 — Extensions and credits (about 3 weeks)

- `ext_defs` and `ext_values` ; file, rule and CeMO sources; the sample-then-estimate flow; query builder support.
- `credit_ledger` , the price list, pools, caps and alerts; the Credits area for owners; credit use for Builders.
- **Done when** a Builder adds a Persona table by asking CeMO, sees the estimate, approves it, and uses it in a skill and a deck, with the credits visible on both sides.

Phase 6 — Workspace onboarding (about 3 weeks)

- The wizard, dump upload, jobs, the loader in steps, editors for brands, relevance and topics, the load report, and Health (followers, links, performance).
- **Done when** data ops onboards a second listening client end to end without an engineer, and Fintech can be re-created from its CMS state with the same counts.

Phase 7 — Learning (about 1.5 weeks, then ongoing)

- Insights with the three-workspace floor, signals on each role's page, the creations roll-up, before-and-after outcomes.

- **Done when** a release that started from client creations is measured four weeks later.

Sequencing notes

- Phases 1 to 3 need only our three workspaces and Fair staff, so they can start now.
- Phase 3 is the bet: if the Role Lab works, every later improvement gets faster, including the rest of this plan.
- Phase 6 depends on the shape of the listening team's daily sync. Until then data ops onboards with the scripts, as today; if a second client signs before phase 6, it moves up.
- Phase 5's credit prices need a month of measured cost per action, which phase 1 starts recording.

Decisions for Refal, and risks

Decisions (4 Oct)

1. **Default follow mode.** *Decided: every client follows the latest release (major releases after a 7-day preview banner), and a release never removes or changes what a client built: their own deck templates (which they version), skills, reports, house rules and extensions.*
2. **Learning across clients.** The plan learns from every workspace by default, anonymised above a three-workspace floor, with a per-workspace switch off. Some clients may require opt-in instead. Either way the terms should say that anonymous patterns of use improve the product while data and text never leave the workspace; clients who feel they built their own version should not be surprised to learn Fair learns from it.
3. **Topic re-labelling.** Keep the listening team's topics and only rename, merge and tag them (planned), or also re-label the catch-all "Others" with our own model in the label cron. The second fixes the 42–59% problem sooner but makes our topics diverge from Fair Listening's.
4. **Codenames.** Chorus (PR), Atlas (Brand & KOL) and Spark (Social Media), versioned major.minor. Decided.
5. **Role owners.** Decided: Audia, Wega and Arneta own all three roles together for now, and Audia and Arneta may also change design. Refal and Rafli are the super admins and the only ones who release.
6. **A Member's creation before approval.** Decided: a Member can create and use their creation themselves; it joins the company version, for everyone, only after the Builder approves.

7. First credit prices and plan tiers. The placeholders stand for now; pricing gets its own discussion once the build is done. Until then the CMS records cost without charging.

Risks

Risk	Why it matters	Mitigation
Too much freedom	Teams hide the tiles that would have warned them; a PR team that mutes "Promo" misses a promo-driven crisis	Locked fields for anything safety-relevant (the ladder, the floor); "hidden for this team" notes stay visible; status still computes on hidden tiles
Model drift	After a year, every workspace is a different product and support cannot reason about it	Company versions hold only changes on a shared Fair role, "versions behind" shown, Fair releases flow to followers, good client creations are promoted into the Fair role
House rules as prompt injection	A rule like "always say sentiment is positive"	Rules placed under fixed rules, refused at save when they ask for numbers or evidence changes, numbers never come from the model
Recompute cost	Relevance and grouping changes rewrite many rows and refresh views	Jobs in slices, per-brand transactions, preview runs on samples, nightly window for full refreshes
Vercel Cron limits	A one-minute job runner needs a plan that allows per-minute crons and enough function time	Confirm the plan; fall back to five-minute polling plus a "Run now" button that triggers a slice directly
Sparse learning	Three workspaces cannot produce anonymous insights	Floor kept; role owners read each workspace's page in the CMS directly until there are more; Fair's own internal workspaces count
Upstream changes	The listening team's daily sync or taxonomy changes under us	<code>data_sources</code> as an adapter, topic mapping table, load reports that flag unrecognised columns

Risk	Why it matters	Mitigation
Data quality hidden by polish	A beautiful dashboard on a gap (no own Instagram after 9 Sep, single follower capture) misleads	Health checks, notes CeMO reads, "unavailable" over a guessed number
A plausible but wrong Lab change	The tests pass, but behaviour drifts somewhere the tests do not look	Every client-reported problem becomes a test; staged rollout; one-click rollback by role owners; release only by Refal or Rafli
Release bottleneck	Two people release everything; proposals pile up	One-screen proposals with test results; CMS Home lists what waits; rollback delegated; a third releaser can be added later
Builder approvals stall	Members wait, then stop creating	Approvals inbox with reminders; CMS Home flags items waiting more than 7 days; several Builders per client
Extension costs surprise a client	An extension CeMO fills costs more with every new row, every day	Sample and estimate first, Builder approval, daily cost visible, monthly caps
Recipes cannot express a need	The Lab stalls on changes that need code	Code requests written up by the Lab with spec and tests; widen the query builder where requests repeat

Open items for the listening team

These came out of loading Fintech and will be the first things the CMS's Health area shows:

- Follower history: one capture per account, so follower growth cannot be shown.
- @gopayindonesia: 57–65 own Instagram posts a day on 7–9 Sep, usually a handful.
- No own Instagram posts captured after 9 Sep.
- Topics: "Others" holds 42–59% of comments; Paylater has 10.